

新论文 PDF 批处理命令

本文档使用 `new_paper_pdf` 作为批次名占位符。运行前，将一篇或多篇 PDF 放入：

```
/data3/paper_analysis/papers_pdf/new_paper_pdf/
```

如需处理其他批次，全文替换 `new_paper_pdf`。每个批次应使用独立的 Markdown、章节、checkpoint 和 repo 目录。

1. 路径约定

```
PDF 输入：      /data3/paper_analysis/papers_pdf/new_paper_pdf
Markdown 输出： /data3/paper_analysis/papers_md/new_paper_pdf
章节输出：      /data3/paper_analysis/paper_secs/new_paper_pdf
Checkpoint：
/data3/paper_analysis/paper_extract_checkpoints/new_paper_pdf
分析结果：      /data3/paper_analysis/repos/repo_new_paper_pdf
笔记预览：      /data3/paper_analysis/temp/new_paper_pdf_notes_preview
```

处理链路：

```
PDF → 全文 Markdown → 图片 OCR 文字原位注入 → 章节 Markdown → Claude 结构化分
析 → 可选的 Obsidian 独立笔记
```

2. PDF 转 Markdown

先检查 Marker 命令，不实际转换：

```
# 批次占位符：new_paper_pdf
# PDF 输入目录名：papers_pdf/new_paper_pdf
# Markdown 输出目录名：papers_md/new_paper_pdf
cd /data3/paper_analysis

python3 scripts/pdf_to_md.py batch \
  /data3/paper_analysis/papers_pdf/new_paper_pdf \
  --output /data3/paper_analysis/papers_md/new_paper_pdf \
  --workers 2 \
  --dry-run
```

正式转换；中断后可用相同命令继续，并跳过已有输出：

```
# 批次占位符:new_paper_pdf
# PDF 输入目录名:papers_pdf/new_paper_pdf
# Markdown 输出目录名:papers_md/new_paper_pdf
cd /data3/paper_analysis

python3 scripts/pdf_to_md.py batch \
    /data3/paper_analysis/papers_pdf/new_paper_pdf \
    --output /data3/paper_analysis/papers_md/new_paper_pdf \
    --workers 2 \
    --skip-existing
```

3. 对 Markdown 中的图片执行 OCR 并原位注入

PDF 转 Markdown 完成后、按章节拆分前，对 Markdown 引用的本地图片逐张执行 OCR：

```
# 批次占位符:new_paper_pdf
# Markdown 输入及回写目录名:papers_md/new_paper_pdf
cd /data3/paper_analysis

python3 scripts/md_image_ocr_injector.py \
    --path /data3/paper_analysis/papers_md/new_paper_pdf \
    --torch-device cuda \
    --cuda-devices 0,1 \
    --workers-per-gpu 2 \
    --marker-root /data3/Projects/marker \
    --marker-python /home/descfly/miniconda3/bin/python3
```

脚本递归扫描目录中的 .md 文件，对每个本地图片引用调用 Marker 单文件转换脚本，并将识别文字以引用块形式插入原图片标签之前，因此 OCR 文字会随图片保留在原文对应位置：

```
> **[图片提取文字 (无描述)]:**
> OCR 识别内容.....


```

脚本默认在修改前生成 <原文件>.md.bak 备份；中断后可重复执行，已有 **图片提取文字** 标记的图片会自动跳过。网络图片和不存在的本地图片只打印提示并保持原文不变。Marker 路径参数与 pdf_to_md.py 使用相同的源码目录和 Python 解释器；也可分别通过 **MARKER_ROOT**、**MARKER_PYTHON** 环境变量覆盖。

示例使用两张 RTX 4090，并按 Marker 对 24GB 显存的估算在每张卡启动 3 个持久 worker；图片会在两张卡之间均匀分片。每个 worker 只加载一次模型，避免逐图重复加载。运行期间终端每 2 秒刷新总完成数、百分比、累计用时及各 GPU 分片进度。

注意：

- 本步骤依赖 PDF 转 Markdown 阶段导出的本地图片，因此不要向 `pdf_to_md.py` 传入 `--disable_image_extraction`。
- `pdf_to_md.py --force_ocr` 用于强制整页 PDF 文字 OCR；本步骤用于对导出的图、表和截图再次 OCR，两者作用不同，可同时使用。
- 不要传入 `--remove-images`；后续 `paper_mdsplit_batch.py` 还需要将原始图片复制到章节目录。
- 该脚本没有 `dry-run`。首次处理新批次时应保留默认备份，并检查终端中的成功、跳过、超时和失败数量。

4. 按章节拆分 Markdown

```
# 批次占位符 : new_paper_pdf
# Markdown 输入目录名 : papers_md/new_paper_pdf
# 章节输出目录名 : paper_secs/new_paper_pdf
cd /data3/paper_analysis

python3 scripts/paper_mdsplit_batch.py \
    /data3/paper_analysis/papers_md/new_paper_pdf \
    /data3/paper_analysis/paper_secs/new_paper_pdf
```

对比三个阶段的论文数量：

```
# 批次占位符 : new_paper_pdf
# 依次检查 : papers_pdf/new_paper_pdf、 papers_md/new_paper_pdf、
# paper_secs/new_paper_pdf
find /data3/paper_analysis/papers_pdf/new_paper_pdf \
    -maxdepth 1 -type f -iname '*.pdf' | wc -l
find /data3/paper_analysis/papers_md/new_paper_pdf \
    -mindepth 1 -maxdepth 1 -type d | wc -l
find /data3/paper_analysis/paper_secs/new_paper_pdf \
    -mindepth 1 -maxdepth 1 -type d | wc -l
```

PROF

`paper_mdsplit_batch.py` 没有 `dry-run`；单篇拆分失败时会打印错误并继续，因此应检查终端输出和目录数量。

5. 逐篇执行结构化分析

先只打印第一篇的选择和 Prompt，不启动 Claude、不写 checkpoint：

```
# 批次占位符 : new_paper_pdf
# 章节输入目录名 : paper_secs/new_paper_pdf
# Checkpoint 目录名 : paper_extract_checkpoints/new_paper_pdf
# 分析输出目录名 : repos/repo_new_paper_pdf
cd /data3/paper_analysis

python3 scripts/run_all_papers.py \
```

```
--paper-base-dir /data3/paper_analysis/paper_secs/new_paper_pdf \  
--checkpoint-dir  
/data3/paper_analysis/paper_extract_checkpoints/new_paper_pdf \  
--output-repo-dir /data3/paper_analysis/repos/repo_new_paper_pdf \  
--limit 1 \  
--dry-run
```

正式顺序处理全部论文：

```
# 批次占位符 : new_paper_pdf  
# 章节输入目录名 : paper_secs/new_paper_pdf  
# Checkpoint 目录名 : paper_extract_checkpoints/new_paper_pdf  
# 分析输出目录名 : repos/repo_new_paper_pdf  
cd /data3/paper_analysis  
  
python3 scripts/run_all_papers.py \  
  --paper-base-dir /data3/paper_analysis/paper_secs/new_paper_pdf \  
  --checkpoint-dir  
/data3/paper_analysis/paper_extract_checkpoints/new_paper_pdf \  
  --output-repo-dir /data3/paper_analysis/repos/repo_new_paper_pdf
```

该步骤会逐篇启动 Claude，并依次执行 `paper-experiment-idea` 和 `paper-knowledge`，可能产生模型调用费用。中断后重新执行完全相同的命令，脚本会根据 `progress.json` 跳过已完成论文。不要让不同批次或不同论文选择共用同一个 `checkpoint` 目录。

输出结构：

```
repo_new_paper_pdf/  
├─ experiment_repo/  
├─ idea_repo/  
└─ knowledge_repo/
```

PROF

6. 可选：拆成 Obsidian 独立笔记

先写入隔离目录检查结果：

```
# 批次占位符 : new_paper_pdf  
# 分析输入目录名 : repos/repo_new_paper_pdf  
# 隔离笔记输出目录名 : temp/new_paper_pdf_notes_preview  
cd /data3/paper_analysis  
  
python3 scripts/repo_mdsplit_batch.py \  
  /data3/paper_analysis/repos/repo_new_paper_pdf \  
  --notes-base /data3/paper_analysis/temp/new_paper_pdf_notes_preview
```

确认后写入正式笔记库：

```
# 批次占位符：new_paper_pdf
# 分析输入目录名：repos/repo_new_paper_pdf
# 正式笔记根目录固定为：/data3/paper_analysis
cd /data3/paper_analysis

python3 scripts/repo_mdsplit_batch.py \
    /data3/paper_analysis/repos/repo_new_paper_pdf \
    --notes-base /data3/paper_analysis
```

该步骤可省略。它不调用模型，但没有 dry-run，并会直接覆盖目标目录中的同名笔记。

7. ISCA26 示例输出

本次为避免混入已有的单篇历史结果，输入沿用 paper_isca26，全量生成目录使用 paper_isca26_full：

```
PDF 输入：          /data3/paper_analysis/papers_pdf/paper_isca26
Markdown 输出：     /data3/paper_analysis/papers_md/paper_isca26_full
章节输出：          /data3/paper_analysis/paper_secs/paper_isca26_full
Checkpoint：
/data3/paper_analysis/paper_extract_checkpoints/paper_isca26_full
分析结果：          /data3/paper_analysis/repos/repo_paper_isca26_full
笔记预览：
/data3/paper_analysis/temp/paper_isca26_full_notes_preview
```

当前 ISCA26 的 172 个 PDF 已全部生成 Markdown 和章节，172 个分析状态均为 DONE。聚合分析位于 repos/repo_paper_isca26_full，目录结构如下：

PROF

```
repo_paper_isca26_full/
├── experiment_repo/
│   ├── 实验_kernel调度.md
│   └── 实验_硬件架构.md
├── idea_repo/
│   └── idea库.md
└── knowledge_repo/
    ├── 知识库_kernel调度.md
    ├── 知识库_硬件架构.md
    ├── 知识库_算法pipeline.md
    └── 知识库_芯片设计.md
```

其中聚合 Markdown 以 ## 标题区分论文或术语。例如：

```
# idea库
```

```
## Accelerating MoE with Dynamic In-Switch
```

- baseline方法是什么？
- 论文方法是什么？如何对应解决Baseline的缺陷？

```
# 实验_硬件架构
```

```
## Accelerating MoE with Dynamic In-Switch
```

- 属于硬件架构的实现是什么？实验比较什么？
- 模拟器名，模拟器链接，或论文修改的模拟器。
- 模拟器模拟什么性能，修改了什么。
- 开源情况及模拟器输入、执行和性能输出过程。

全量运行后，同一类聚合文件会继续增加其他 ISCA26 论文的 ## <论文标题> 条目；

repo_mdsplit_batch.py 再将这些条目拆成单独的 Obsidian 笔记。